

ROBOTIC PROCESS AUTOMATION

UNIT-1

(RPA Foundations & RPA Skills)

What the Technology Can Do (RPA)

1. Introduction to RPA Investment

- RPA (Robotic Process Automation) is one of the hottest categories for venture capital investment.
- Example investments:
 - **Automation Anywhere (Nov 2018):**
 - Softbank Vision Fund invested **\$300 million**.
 - **Series A round total: Over \$500 million.**
 - Included strategic investors like **Workday** (top cloud-based ERP operator).
 - Workday Ventures cohead's note: "RPA is becoming a bigger focus for our customers, which is why it's important for us to partner closely with Automation Anywhere, a market leader."

2. UiPath Investment & Growth

- **UiPath (April 2019):**
 - Raised **\$568 million** for its Series B round.
 - Investors: **Coatue, Wellington, CapitalG, Accel, Sequoia** (tier-one investors).
- **Growth within 2 years:**
 - Valuation: **\$110 million → \$7 billion**.
 - Annual recurring revenues: **\$8 million → \$200 million**.
 - Employee base: **16× increase to 2,500 employees**.
 - **6 new releases** of the UiPath Enterprise RPA platform launched.
 - Secured large customers: **McDonald's, Duracell, Google, Nippon Life Insurance, Ricoh, Voya Financial**.

3. UiPath CEO's Statement (Daniel Dines)

- "We are at the tipping point. Business leaders everywhere are:
 - Augmenting their workforces with **software robots**.
 - Rapidly accelerating the **digital transformation** of businesses.
 - Freeing employees to spend time on **more impactful work**."
 - UiPath's Vision:
 - **Democratize RPA** – make it accessible to everyone.
 - Deliver on the vision of **a robot helping every person**.
 - **Platform unlocks true potential of AI** and other emerging technologies.
 - Supported by customers, partners, and investors – motivation to evolve further.
 - Quote: "We are just getting started."
-

What Is RPA?

1. Origin of the Term RPA

- The term **RPA (Robotic Process Automation)** was coined in **2012** by **Pat Geary**, Chief Evangelist for **Blue Prism**.
- The category was still evolving at that time, hence the term can feel **slippery/confusing**.

2. Breaking Down the Term “RPA”

- **Robotic:**
 - Does **not** mean a physical robot.
 - Refers to a **software-based robot (bot)** that automates human actions in the workplace.
 - Mainly used in **clerical and administrative (white-collar) functions**.
 - Delivered either via **cloud** or **downloadable software**.
 - The use of the word "robotic" is partly a **savvy marketing move** (robots are seen as “cool”).
- **Process:**
 - Not very descriptive.
 - A better word would be **“tasks”** (individual action items that are part of a process).
- **Automation:**
 - This part of the term is **spot-on**.
 - It represents the **core functionality** of RPA: automating tasks.

3. Definition of RPA (In Simple Terms)

- RPA involves **bots** that perform a set of **specified tasks**, such as:
 - Cut-and-paste of information between applications.
 - Opening a website and logging in.
 - Opening an email and attachments.
 - Reading/writing a database.
 - Extracting content from forms/documents.
 - Performing calculations and workflows.
- These tasks may seem **mundane/boring**, but that’s the point:
 - RPA focuses on tasks that **waste human effort**, so employees can do **more important work**.

4. How Vendors Define RPA

- **UiPath:**
 - RPA allows anyone to configure software (“robots”) to **emulate and integrate human actions**.
 - Robots use the **UI (user interface)** to capture data and manipulate applications like humans.
 - They interpret, trigger responses, communicate with other systems.
 - Benefits: **Never sleep, make zero mistakes, cost less than employees**.
- **Automation Anywhere:**
 - RPA is **simple yet powerful**.

- Provides tools to **create software robots** for automating business processes.
- Bots = **Digital Workforce**.
- Features: configurable, code-free, non-disruptive, scalable, easy to control.
- Bots can **interact with systems, learn, and be cloned**.
- **PEGA:**
 - RPA = **fast, low-risk starting point** for automation.
 - Useful for processes dependent on **legacy systems without APIs**.
 - Ensures **faster and efficient outcomes**.
 - Limitation:
 - RPA **does not transform an organization by itself**.
 - Not a fix for **broken enterprise-wide processes**.
 - For that → requires **end-to-end intelligent automation**.
- **Kryon Systems:**
 - RPA creates **true virtual workforces**.
 - Virtual workforce = **software robots** that execute tasks on enterprise apps.
 - Managed like a **normal team** in the organization.
 - Virtual workers interact with people like employees.
 - Advantages: Faster, more accurate, lower cost.
 - Impact: Strong business outcomes + **significant ROI**.

5. General View

- Vendors provide **different definitions and approaches**, sometimes **starkly different**.
- Common idea: RPA is like having a **digital worker**.
- Uses technologies like **screen scraping and workflows** to copy daily employee tasks.
- Fun fact: Some companies even **name their bots**!

6. RPA vs. Other Automation (e.g., Excel Macros)

- **Excel Macros:**
 - Limited to **a particular application**.
 - **RPA:**
 - Works across **multiple systems** (PCs, mainframes, etc.).
 - Can **record human actions** to help create automation.
 - Some RPA systems use **AI (Artificial Intelligence)** for advanced capabilities.
 - Over time, RPA platforms become a **knowledge repository** about how work is done.
 - This repository provides **insights** to improve workflows and processes, driving **further efficiency**.
-

Flavours of RPA

1. Reason for Different Flavours

- RPA has **different approaches** because:
 - The technology is **still evolving**.
 - Vendors try to **redefine RPA** to stand out in the marketplace.

2. Main Flavours of RPA

On a high level, RPA can be divided into **three main types**:

(a) Attended RPA

- Also known as: **Robotic Desktop Automation (RDA)**.
- **First form of RPA** (around **2003**).
- Works in **collaboration with a human** for certain tasks.
- **Example**:
 - Call centre representative speaks to a customer.
 - At the same time, RPA software **looks up information**.
- Best for **front-office functions** where humans + bots work together.

(b) Unattended RPA

- **Second generation of RPA**.
- Runs **without human involvement**.
- Bot is **event-triggered** (e.g., a customer sends an invoice via email).
- Best suited for **back-office functions**.

(c) Intelligent Process Automation (IPA)

- Also known as: **Cognitive RPA**.
- **Latest generation** of RPA.
- Uses **AI (Artificial Intelligence)** to:
 - **Learn over time**.
 - **Interpret documents** (e.g., invoices).
 - Make decisions using its own **insights and judgments**.
- Result: Requires **less human intervention** compared to earlier generations.

3. Importance of Understanding Flavours

- Some RPA systems **specialize in a specific approach**.
 - When adopting automation, organizations should:
 - Analyse **which type of RPA suits their needs best**.
-

History of RPA

Ancient Roots of Automation

- First mention in Homer's The Iliad: Hephaestus (Greek god of blacksmiths) used automatons (machines) to build weapons for the gods.

Modern Development (last 70 years)

- **Mainframe Era:** IBM's large, expensive machines. Used for payroll, customer accounts. Innovators like Ross Perot offered outsourcing for affordability.
- **PC Revolution:** Intel's microprocessor + Microsoft's OS. Allowed automation for all businesses (word processors, spreadsheets). Drawbacks: complex IT environments, costly integrations, custom coding, repetitive data moving → sometimes reduced productivity.

Emergence of RPA (Early 2000s)

- Screen scraping (automating data movement among apps) improved efficiency.
- Early RPA seen as low-tech/commodity. Investors focused more on cloud computing.

RPA Inflection Point (~2012)

- Post-financial crisis: Companies wanted cost-cutting, ERP matured.
- Needed faster digital transformation to avoid disruption from tech firms.
- Banking and regulated industries: reduced paperwork, improved audits, security, control.
- RPA became more sophisticated, easier to use, higher ROI.
- Large companies started mission-critical adoption.
- Millennials entering workforce: wanted careers and engaging work, not repetitive tasks.
- Mihir Shukla (CEO, Automation Anywhere): RPA evolution = gradual progress with periodic breakthroughs (like iPhone after years of incremental innovation).

Present Growth of RPA

- Fastest growing segment in software industry.
- Gartner: 2018 spending \$850M (up 63% YoY), forecast \$1.3B in 2019.
- Transparency Market Research: Global RPA market projected \$5B by 2020.

Key Metrics & Projections

- By 2020: RPA + AI → reduce business shared-service centers by 65% (Gartner). Adoption: 40% of large enterprises (vs 10% in 2019).
 - By 2023: Market saturation expected (Deloitte). Forecast \$12B spending on RPA services (Forrester).
 - By 2025: Financial impact may reach \$6.7T (McKinsey).
 - Regional share: North America 51%, Western Europe 23%, Asia (esp. Japan) growing.
-

The Benefits of RPA

1. ROI (Return on Investment)

- Strong ROI compared to other enterprise software.
- Computer Economics Technology Trends 2019: Survey of 250 companies.
 - 12% implemented RPA.
 - 50% achieved positive ROI within 18 months; others mostly breakeven.
- David Wagner (VP Research, Computer Economics): RPA adoption growing due to early success.
- A.T. Kearney:
 - Software robot costs $\frac{1}{3}$ of offshore employee, $\frac{1}{5}$ of onshore staff.
 - Barclays Bank: savings = 120 FTEs + \$250M reduction in bad debt provisions.
 - Telefónica O2: 160+ robots, 15 core processes, 500k+ transactions/month, ROI > 650%.

2. Impact of Small Improvements

- Even 10–20 seconds saved per task (e.g., copy-paste) scales significantly across global organizations.
- Many companies track total hours saved via RPA as ROI.
- Forrester Survey: 86% respondents saw efficiency gains from RPA.

3. Relative Ease of Implementation

- Unlike CRM/ERP, RPA does not need heavy integration or coding.
- Sits on top of existing IT systems.
- Easy for employees → less dependence on IT department.
- Reduces training needs.
- Quicker achievement of objectives + IT can focus on higher priorities.
- Important due to ongoing trend of lower IT investment.

4. Compliance

- One regulatory violation can threaten company survival (e.g., Enron, Theranos).
- Employees may make mistakes, but RPA bots can be pre-configured for compliance.
- Many RPA vendors support key laws:
 - Sarbanes–Oxley Act (SOX)
 - GDPR (General Data Protection Regulation)
 - HIPAA (Health Insurance Portability and Accountability Act, 1996)
- NICE Survey: Compliance improvement = top benefit from RPA.
- Reduces human data handling → lowers fraud risk.
- Provides strong audit trail for monitoring & tracking.

5. Customer Service

- Customers expect speed + accuracy, but companies face high contact volumes.
- RPA ensures all steps are followed at scale → better customer satisfaction.
- Improves Net Promoter Score (NPS).
- Mihir Shukla (Automation Anywhere CEO): RPA can reduce mortgage processing time from 15 days → 7 minutes.

6. Employee Satisfaction

- Employees spend less time on repetitive tasks.
- Results: higher productivity, reduced turnover.
- Forrester Survey: 72% respondents reported higher employee engagement due to RPA.

7. Wide Application

- RPA applies across multiple functions: legal, finance, HR, marketing, sales, etc.

8. Data Quality

- Reduced human error → improved accuracy.
- More data generated due to scalability.
- Larger, more robust datasets for analytics + AI.

9. Digital Transformation

- Priority for CEOs, but legacy systems are expensive to replace.
- RPA integrates with legacy systems easily → faster + cheaper transformation.

10. Scalability

- Meeting sudden demand increase = difficult with human hiring.
 - RPA bots are cheaper and faster to scale compared to employees.
-

The Downsides of RPA

- **Cost of Ownership:** Business models vary (subscription, multiyear license, per bot charges). Additional costs include training, ongoing maintenance, other required software/hardware, and third-party consultants for implementation.
- **Technical Debt:** RPA is not a comprehensive solution. Requires reworking, updates, and changes as business processes evolve. Bots may fail if processes change, so continuous attention is needed.
- **Enterprise Scale:** Scaling RPA across the organization is difficult. Managing numerous bots and ensuring IT collaboration is challenging. Deloitte UK study: Only 3% of organizations scaled RPA to 50+ bots.
- **Security:** Growing risk as RPA covers mission-critical processes. A breach can expose sensitive data. In manufacturing, risks may include property damage or bodily harm, especially with attended RPA.
- **Expectations:** Hype leads to unrealistic expectations. PEGA survey: Average time to develop a quality bot = 18 months; only 39% deployed on time.
- **Preparation:** Without analysing current tasks deeply, RPA may automate bad processes, leading to inefficiency.
- **Limits:** RPA mainly handles routine, repetitive tasks. Judgment-based tasks (approvals, verifications) still need humans. With AI integration, these limitations may reduce (e.g., insurance claim adjudication).
- **Virtualized Environments:** RPA may fail in remote-access platforms (e.g., Citrix) since it cannot capture on-screen text. Latest RPA tools (like UiPath) are addressing this issue.

RPA Compared to BPO, BPM, and BPA

1. BPM (Business Process Management)

- Origin: Emerged in response to global competition (1970s–1980s). US companies adopted **Six Sigma**, **Lean Production (Toyota principles)**, **TQM** (blend of Six Sigma + Lean), and computer technologies like FileNet (digital workflow, later acquired by IBM). ERP vendors (e.g., PeopleSoft) also emerged.
- BPM = comprehensive improvement of business processes (optimizing both employee systems + IT assets).
- Business Process Management Software (BPMS) like **Laserfiche** (founded 1987 by Nien-Ling Wacker using OCR for document search).
- Difference from RPA:
 - BPM = changing **entire processes**, requires detailed documentation, training, and takes more time/effort.
 - RPA = automates **tasks**, quicker, less effort.
- BPM attractive for heavily regulated industries (financial services, healthcare). Risk: too much structure may stifle innovation.
- RPA is **complementary** to BPM → BPM improves processes, RPA fills gaps.
- UiPath analogy:
 - BPM = rebuilding roads for self-driving cars.
 - RPA = driving existing cars like humans.
 - Both needed, depending on situation.

2. BPO (Business Process Outsourcing)

- Definition: Outsourcing business functions (payroll, HR, procurement, customer support).
- Market Size: Forecasted revenue **\$343.2B by 2025** (Grand View Research).
- Top Players: ADP, Accenture, Infosys, IBM, TCS, Cognizant.
- Advantages:
 - Lower wages in other countries (**labour arbitrage**).
 - Access to educated, multilingual workforce.
 - Focus on core business while specialists handle non-core functions.
- Types:
 - **Offshore** – employees in distant countries.
 - **Nearshore** – neighbouring country, higher cost but easier collaboration.
 - **Onshore** – same country, but wage differences apply.
- Drawbacks:
 - Quality issues (e.g., poor communication with call center agents).
 - Security risks when handling sensitive company data.
 - Rising labor costs in countries like China & India → relocation disruptions.
 - Political instability can disrupt BPO operations.
- RPA Connection:
 - RPA automates BPO-type activities.
 - Studies (Everest Group, KPMG, Deloitte): RPA offers up to **70% cost advantage** vs outsourcing.
 - BPO firms themselves are **heavy users of RPA** to stay competitive.

3. BPA (Business Process Automation)

- Definition: Use of technology to automate a **complete process**.
- Example: Employee Onboarding – multiple repeatable steps with paperwork → BPA streamlines at scale.
- Difference from RPA:
 - **RPA** = automates **part of a process** (specific tasks).
 - **BPA** = automates **entire process** end-to-end.

Aspect	RPA (Robotic Process Automation)	BPM (Business Process Management)	BPO (Business Process Outsourcing)	BPA (Business Process Automation)
Definition	Software robots automate repetitive tasks	Improves & optimizes entire business processes	Outsourcing business functions to third-party vendors	Automating a complete process using technology
Focus	Task-level automation	Process-level optimization	Cost-saving & focus on core business	End-to-end process automation
Change Needed	Minimal (works on existing systems)	Requires process redesign & documentation	Requires contracts, vendor management	Requires full integration of tools
Speed of Implementation	Fast (weeks/months)	Slow (months/years)	Medium (depends on vendor setup)	Medium to long (integration-heavy)
Cost Factor	High-cost savings (up to 70% vs outsourcing)	Expensive, but long-term efficiency	Cost advantage via labour arbitrage	Costly at start, but scalable
Use Cases	Data entry, report generation, invoice processing	End-to-end workflows, compliance, risk management	Payroll, HR, customer support, procurement	Employee onboarding, full HR cycle automation
Industries	Finance, healthcare, insurance, IT services	Financial services, healthcare (regulated industries)	IT, HR, customer service, back-office ops	Any industry with repetitive full workflows
Dependency	Works with existing IT & BPM systems	Requires new workflows & employee training	Relies on external workforce	Relies on integrated tech stack
Risk	Low risk (non-intrusive)	Too much structure may reduce innovation	Security risks, quality issues, political instability	Integration complexity
Relation with RPA	—	Complementary (RPA fills gaps BPM misses)	RPA is replacing/reducing BPO need	RPA = part of BPA (task automation inside full process)

Consumer Willingness for Automation

Introduction

- Automation of consumer-facing activities (like **chatbots on smartphones or websites**) is becoming increasingly common.
- Key questions:
 - Does this technology provide a **good experience**?
 - Could it be doing **more harm than good**?

Challenges

- Many automation technologies are **not perfect**, especially in **complex environments**.

Case Study: Helpshift Report

- Helpshift is an AI-based **digital customer service platform**.
- It automates **80% of customer support issues** for big D2C (Direct-to-Consumer) brands such as:
 - Flipboard
 - Microsoft
 - Tradesy
 - +60 other companies
- The report is based on analysis of:
 - **75 million customer service tickets**
 - **71 million bot-sent messages**

Findings from the Report

1. **55% of respondents** – and **65% of millennials** – prefer chatbots in customer service if:
 - They are **more efficient**
 - They **reduce phone time** required to resolve an issue or explain a problem
2. **49% of respondents** appreciate the **24/7 availability** of chatbots

Current Situation

- **Chatbot technology** is still in **early stages**
- Can sometimes be **glitchy** or even **annoying**
- However, in future:
 - This automation will become **more important**
 - It will also become part of the **RPA roadmap**

Expert View

- **Linda Crawford (CEO of Helpshift):**
 - “Since the majority of Americans **dread contacting customer support**, there is a huge opportunity for chatbots to:
 - Fill the gap
 - Improve customer support experience for both **consumers and agents**.”

The Workforce of the Future

The modern concept of work can be traced back to **Henry Ford**, who revolutionized the automotive industry by scaling operations and introducing new policies. To attract the best employees, he **doubled wages to \$5 a day**, which expanded the middle class. He also introduced the **40-hour workweek with weekends off**, setting a foundation for modern work culture.

Since then, the fundamentals of work have not changed much. Although the **gig economy** (services through Uber, Lyft, etc.) has emerged, the structure of **office work** remains durable. However, many activities are still tedious and repetitive. According to **McKinsey Global Institute**, white-collar workers spend **60% of their time on manual tasks** like emails, spreadsheets, notes, and calls. Inefficient processes waste nearly **\$575 billion annually in the US** (David White).

Here, **Robotic Process Automation (RPA)** is expected to have a significant impact because repetitive processes will be automated. A concern is **job loss**: a **Forrester survey (2025)** predicts that **9% of the world's jobs (230 million)** will be lost due to software automation. The study also estimates **16% job replacement in the US** with **only 9% new jobs created**. Similarly, **McKinsey & Company** analysis shows that technologies like RPA could **automate 45% of workforce activities**.

Interestingly, CEOs usually avoid directly addressing job losses during automation projects, since it may cause fear and negative headlines. Instead, they highlight the overall benefits of transformation. But in reality, the **impact will grow significantly as RPA systems become more robust**. History shows that whenever technology causes job losses (e.g., **Industrial Revolution**), it leads to political and regulatory changes, and sometimes technology becomes a target of criticism.

Although companies generally avoid layoffs because they are costly and harmful to organizations, managers will increasingly need to **reskill employees and find new roles** to handle automation-driven changes.

Overall, automation has the potential to create a **better society**, where workers focus on **interesting and engaging activities** instead of repetitive tasks. It will also encourage **continuous renewal of knowledge and skills**. However, **proactive efforts** from both companies and governments are necessary to ensure a smoother transition.

A survey by **PEGA** shows that nearly **70% of people believe the future workforce will consist of both humans and machines**, and **88% are comfortable with this**, as long as they are **not managed by machines**.

RPA Skills

RPA does not require deep programming skills, but there is a strong need to understand certain high-level technology concepts. The technological ecosystem often appears overwhelming because of the large number of technical acronyms such as ACL, API, OCR, CPU, HTTP, IP, JSON, NOC, PCI, RAM, and SaaS. Even professionals with years of experience in the technology industry may not know many of these terms in detail or may only have a vague understanding of their meanings. For instance, Kubernetes, also referred to as K8s, is defined as an open-source system designed to automate deployment, scaling, and management of containerized applications. However, this definition itself is quite complex and requires a deep knowledge of computer systems and software architecture to fully grasp.

Although the world of technology is filled with such complex terminologies and frameworks, the encouraging fact is that for using Robotic Process Automation effectively, one does not need to master all these advanced or highly technical concepts. Instead, there are only a handful of important terms and concepts that need to be understood in order to work with RPA successfully. These essential concepts form the foundation of practical knowledge that professionals should focus on, rather than attempting to become experts in every aspect of software or systems architecture.

Thus, the key point is that while RPA operates in a technology-rich environment filled with acronyms and technical jargon, it does not demand deep technical specialization. A learner or practitioner only has to develop familiarity with a select set of terms that directly relate to the working and deployment of RPA tools. This makes RPA accessible to non-programmers and business users, opening the door for wider adoption of automation in various industries.

On-Premise vs. the Cloud

Introduction

The deployment of technology in organizations has traditionally revolved around two major approaches: on-premise systems and cloud computing. Both models have their advantages and limitations, and understanding them is crucial for implementing automation technologies such as RPA, which often depend on the underlying infrastructure.

On-Premise Systems

On-premise technology refers to a model where a company purchases and sets up its own hardware and software within its own data centre. The benefits of this approach include complete control over data, systems, and processes, which is especially important for highly regulated industries requiring high levels of security and privacy. Another advantage is the ability to customize solutions based on unique company needs and IT policies. However, on-premise systems have significant drawbacks. The cost of ownership is one of the biggest challenges since it involves heavy up-front capital investment along with continuous expenses for maintenance, upgrades, and monitoring. IT departments also spend much of their time managing non-core activities rather than focusing on innovation. Additionally, point applications such as Excel may result in fragmented environments where data becomes scattered and difficult to centralize.

Evolution of Cloud Computing

Cloud computing emerged as a response to the limitations of on-premise systems. Interestingly, the concept of cloud traces back to the late 1950s with John McCarthy's invention of time-sharing, which allowed multiple users to access mainframes. However, during the 1980s and 1990s, the on-premise model became dominant because of the widespread adoption of PCs, while the Internet was still underdeveloped and limited to the military, universities, and large organizations. As the Internet matured, cloud computing gained momentum. One of the pioneering examples was Salesforce.com, which offered business applications directly through a browser under the Software-as-a-Service (SaaS)

model. According to Marc Benioff, the cofounder and CEO of Salesforce, the vision was to simplify software delivery, eliminate the need for complex installations, and replace multimillion-dollar software packages with pay-per-user, per-month services delivered instantly via the Internet. Although initially challenging to implement, Salesforce.com grew into the dominant cloud applications company, with revenues of more than \$13 billion in 2020 and a market capitalization of \$131 billion.

Advantages and Disadvantages of Cloud Computing

The cloud model offers several benefits. It eliminates heavy up-front costs, provides scalability, reduces the burden of maintenance, and allows quick deployment of software. However, there are downsides as well. Since companies have less control over the platform, there are vulnerabilities related to security and privacy. Outages are another major concern, as they can cause significant disruption. Cloud services are also not always cheap, with Salesforce.com often criticized for high costs. Nevertheless, cloud computing continues to expand rapidly. Gartner predicted that the global market for public cloud services would grow from \$214.3 billion in 2019 to \$331.2 billion in 2022, almost triple the growth rate of overall IT services.

Amazon Web Services and Other Platforms

A major turning point in cloud evolution was the launch of Amazon Web Services (AWS) in 2005. AWS provided a platform that enabled companies to build cloud-native applications while handling complex infrastructure requirements such as storage, security, compute, database access, analytics, IoT, and deployment. This simplified the development of applications, reduced costs, and introduced a per-use pricing model. The scale of AWS adoption has been massive, with companies like Slack planning to spend \$250 million and Pinterest allocating \$750 million over several years. Other technology giants such as Microsoft and Google developed their own platforms, Azure and Google Cloud, respectively. Many companies now adopt multi-cloud strategies to ensure redundancy and flexibility.

Types of Cloud Models

Cloud computing can be categorized into different approaches. Public cloud refers to services accessed from remote servers owned by providers like AWS, Salesforce, and Microsoft. These services operate on a multitenant architecture that allows secure sharing of large infrastructures, making them cost-effective. Private cloud refers to a model where a company owns and manages its own data center, giving it greater control and security but at the expense of scale and cost advantages. Hybrid cloud combines both models, with the public cloud handling less critical functions and the private cloud managing sensitive processes.

Implications for RPA

For RPA, the choice between on-premise and cloud environments carries significant implications. RPA platforms traditionally started as on-premise solutions and only recently began transitioning to the cloud. The challenge arises because RPA requires deep integration across multiple applications and environments, making it more difficult to design cloud-native systems. In some cases, on-premise RPA systems may be hosted on cloud platforms like AWS, but these are not truly cloud-native since they still require upgrades and maintenance. Therefore, the evolution of cloud-native RPA systems is ongoing and remains a critical factor in the future of automation.

Web Technology

Introduction

The development of the World Wide Web marked a turning point in communication and information sharing. It was created by British scientist Tim Berners-Lee in 1990. His main contribution was the use of hyperlinks that allowed users to navigate across different web pages. Although the foundation was laid in 1990, the real growth of the web came in the mid-1990s with the launch of the Netscape browser, which triggered the Internet revolution.

HTML and Its Role

At the core of web technology was HTML, or Hypertext Markup Language. HTML consists of a set of commands and tags used to display text, apply colors, and present graphics on a web page. Its simplicity was the main reason for the rapid spread of websites, as even beginners could learn and use it easily. For example, commands in HTML work by surrounding content with tags such as `<strong>This is a Title</strong>` which results in bold text. This made it possible for non-technical users to create web pages quickly.

Limitations of HTML

While HTML was effective in its early days, it was limited in design and functionality. As the demand for more interactive and visually appealing websites grew, it became clear that HTML alone was not sufficient. It lacked advanced styling, design flexibility, and the ability to handle complex tasks beyond simple text and graphics display.

Emergence of CSS

To overcome these limitations, CSS or Cascading Style Sheets were introduced. CSS allows developers to control the look and feel of a website. It can define colors, borders, shadows, fonts, spacing, and even create animations. CSS separates the structure (HTML) from the presentation, which made web design more organized and professional. It also allowed multiple pages to share a consistent design without rewriting the same formatting rules.

Role of JavaScript

Another key advancement was the introduction of JavaScript. JavaScript added interactivity and functionality to websites, enabling actions such as validating forms, performing calculations, and creating dynamic content without reloading a page. It made websites more engaging, user-friendly, and capable of handling sophisticated tasks. Over time, JavaScript evolved into a powerful programming language used for front-end as well as back-end development.

Relevance to RPA

Robotic Process Automation (RPA) interacts with web technologies to perform automation tasks. Since many business applications run on web-based systems, RPA needs to recognize and interact with HTML tags, CSS styles, and JavaScript elements to carry out actions like filling forms, extracting data, and clicking buttons. Understanding these technologies helps in building more reliable automation processes.

Programming Languages and Low Code

Introduction to Programming Languages

A programming language is a formal way of giving instructions to a computer so that it can perform specific tasks. These instructions are written in a structured format that the computer can understand and execute. Programming languages generally use simple and meaningful words like **IF, DO, WHILE, and THEN** to make logical decisions and control the flow of operations. However, beyond these simple commands, programming can become complex, especially when dealing with advanced concepts like **object-oriented programming (OOP)**. OOP introduces features such as classes, objects, inheritance, and polymorphism, which allow for building scalable and reusable programs but also add to the complexity of coding.

Popular Programming Languages

In the present digital world, some programming languages dominate because of their efficiency, community support, and versatility. **Python** is highly popular for its simplicity and extensive libraries in data science, artificial intelligence, and automation. **Java** is widely used in enterprise applications, mobile app development, and large-scale systems. **C++** is known for performance-heavy applications like gaming, simulations, and embedded systems. **C#** is the backbone of Microsoft's .NET framework, suitable for building desktop and web applications. **Ruby** is valued for its elegant syntax and is often used in web development with the Ruby on Rails framework. These languages form the backbone of modern software and application development.

Programming in RPA Systems

When it comes to **Robotic Process Automation (RPA)**, programming is required but not at the same depth as traditional software development. RPA platforms are designed to simplify the coding effort and allow users to automate workflows without writing extensive code. This is where the concept of **low code** comes in.

Low Code Approach

Low code is a method where automation can be created with **minimal manual coding**. Instead of writing long scripts, RPA systems provide **drag-and-drop tools, flowcharts, and visual designers** that allow the user to build bots in a more intuitive way. For example, instead of coding an instruction to open an application and copy data, the user can select ready-made components from a toolbox and connect them visually to define the workflow. This reduces the learning curve for non-programmers and speeds up automation development.

Training and Understanding Workflows

Although low code reduces complexity, it does not mean there is no learning involved. To use low code efficiently, one must still understand **workflow structures, process logic, and decision-making steps**. This includes concepts like loops (repeated actions), conditions (IF statements), and exception handling (managing errors). A clear understanding of business processes is also essential because RPA bots need to replicate the steps followed by humans in an organization.

Summary

Thus, programming languages form the base for instructing computers, while low code in RPA simplifies automation development by using visual tools and minimal manual coding. However, proper knowledge of workflows and logical structures remains necessary for effective use of low code platforms.

OCR (Optical Character Recognition)

Introduction to OCR

OCR (Optical Character Recognition) is a crucial feature in RPA platforms, designed to convert printed, scanned, or handwritten documents into machine-readable text. It essentially involves two parts: a scanner device (such as a document scanner or even a smartphone camera) and specialized software that can analyze and recognize the text. By doing this, OCR allows images, PDFs, and paper-based content to be converted into editable and searchable data, making it highly useful for automation tasks.

Challenges in OCR Accuracy

OCR, while powerful, has historically faced many challenges that affect its accuracy. These challenges include:

- **Font size issues** where very small or large text becomes difficult to recognize.
- **Shape of the text** such as decorative or unusual fonts that make recognition harder.
- **Skewness of documents** where text may be rotated or slanted, confusing the OCR engine.
- **Blurred or degraded text** often seen in old documents or low-quality scans.
- **Background noise** such as smudges, stains, or overlapping marks.
- **Language diversity** since OCR must adapt to different writing systems and scripts.

Advancements in OCR Technology

In the early years, OCR systems were far from accurate and often produced errors. However, advances in **AI algorithms, fuzzy logic, and more powerful computing hardware** have dramatically improved OCR's performance. Modern OCR solutions can now achieve accuracy rates close to 100% in many cases, making them reliable for critical business applications.

OCR in RPA Workflows

Within RPA, OCR plays a transformative role by enabling bots to recognize and interact with text-based information across multiple applications. For instance, when recording a person's actions on an application, OCR enhances the bot's ability to capture words and visuals on the screen. This ensures that even if the layout or location of elements changes, the RPA system can still identify them correctly, thereby maintaining the workflow.

Document Processing with OCR in RPA

Automation often deals with large numbers of documents, making OCR indispensable for speeding up processing. A practical example is **loan processing**:

- OCR extracts data such as a person's financial background, property details, and other required information.
 - The RPA system then applies business rules and workflows, such as risk assessment, compliance checks, and routing documents to relevant departments or regulatory agencies.
- This process greatly reduces manual effort, improves accuracy, and accelerates turnaround times.

Limitations of Built-in OCR in RPA Systems

Although many RPA systems include their own OCR capabilities, these are not always sufficient for industries heavily dependent on handwritten or complex documents. Sectors like **healthcare, insurance, government, and banking** still rely on forms that are time-consuming and difficult to process with standard OCR tools.

Advanced OCR Solutions for RPA

To address these limitations, advanced OCR systems like **HyperScience** have emerged. HyperScience uses **machine learning (ML) algorithms** to handle complex recognition tasks, including cursive handwriting, structured field detection in invoices, and data reconciliation.

Such systems are highly efficient, with the ability to **automate up to 90% of document processing**, significantly reducing costs and improving efficiency.

DATABASES

Introduction

At the core of most modern applications lies the concept of a **database**, which is essentially a system designed to store data that can later be searched, updated, and retrieved as needed. Typically, this data is organized into **tables consisting of rows and columns**, providing a structured way of handling information. Among the different types of databases, the **relational database** has emerged as the most dominant form since its invention in 1970 by IBM researcher **E. F. Codd**. This model is based on structured data and is accessed through **SQL (Structured Query Language)**, a scripting language designed to be relatively easy to learn and use.

Commercialization of Relational Databases

Although relational databases were proposed in the early 1970s, their **commercialization began in the late 1970s**, with **Oracle** leading the pioneering efforts. Following Oracle's success, several start-ups also entered the field to capitalize on this opportunity. By the **late 1980s**, the enterprise market was dominated by **Oracle and SAP**, while **Microsoft** became the standard choice for the mid-market segment. This marked the era of widespread adoption of relational databases across industries.

Challenges with Relational Databases

Despite their effectiveness, relational databases came with certain **limitations** that created long-term challenges for organizations. One major issue was **data sprawl**, which refers to the uncontrollable growth and proliferation of tables across an organization. This fragmentation made it difficult to centralize data, thereby preventing businesses from obtaining a **holistic, 360-degree view of their information**. Another challenge was the **high cost** of relational databases, making them less feasible for smaller organizations. Furthermore, with the rise of **cloud computing, real-time applications, and mobile technologies**, relational databases struggled to efficiently handle the increasing demands for data processing and scalability.

Emergence of Alternatives: Data Warehousing and Hadoop

In response to these challenges, **alternative approaches** to data management emerged. One key innovation was the **data warehouse**, which began as an open-source project in the late 1990s by **Doug Cutting**. This effort eventually evolved into **Hadoop**, a technology capable of handling massive volumes of **Big Data**. Yahoo! was one of the first major adopters of Hadoop, using it to manage the data demands of its large-scale digital platforms. Following Yahoo!, companies such as **Facebook and Twitter** also adopted Hadoop. The core advantage of data warehouses was that they enabled organizations to achieve a **comprehensive, 360-degree view of their data**, making large-scale analytics feasible. Over time, investments from companies such as **Google, Amazon, and Microsoft** further advanced data warehouse systems. Additionally, innovative start-ups like **Snowflake** have entered the market, pushing new boundaries in efficiency, scalability, and integration.

New Approaches to Databases

Alongside data warehouses, **new open-source relational systems** such as **MySQL** (now owned by Oracle) and **PostgreSQL** also emerged. However, despite their technical strengths, these systems did not achieve significant traction within large enterprises. The real disruption came from the rise of **next-generation NoSQL databases**.

NoSQL Databases

The **NoSQL model** represented a dramatic departure from traditional relational approaches. Initially launched as an open-source project, NoSQL rapidly gained momentum. A notable example is **MongoDB**, which has seen adoption from **14,200 customers across 100**

countries, with more than **70 million downloads** worldwide. Unlike relational databases that enforce rigid structures, NoSQL offers **flexibility** through its **document model**, making it highly scalable and suitable for handling **petabyte-scale data**. This adaptability has allowed NoSQL databases to thrive in environments requiring massive, unstructured, or semi-structured data management.

Databases in the Cloud

Another major trend reshaping the database industry is the **migration of databases to the cloud**. According to research from **Gartner**, about **75% of databases are expected to be migrated to cloud platforms**. The shift to the cloud provides organizations with several advantages, including easier scalability, cost efficiency, and enhanced capabilities for **analytics and Artificial Intelligence (AI)** integration. Cloud-based databases also reduce infrastructure management overheads, making them an attractive choice for businesses of all sizes.

Future Outlook

Looking ahead, database technology is set to witness **continuous innovation**, driven by cloud platforms, AI integration, and the need to manage massive, complex datasets. Nevertheless, **relational databases will continue to remain the majority choice for companies**, which means that **Robotic Process Automation (RPA)** will continue to interact predominantly with relational systems. However, the growing importance of NoSQL, cloud databases, and advanced data warehouses indicates a **hybrid database environment**, where different types of databases coexist to meet diverse organizational needs.

APIs (Application Programming Interfaces)

Definition and Basic Concept

An **API (Application Programming Interface)** is software that connects two applications together and allows them to communicate. It acts like a bridge between systems. For example, if you want to create a weather app, instead of manually collecting data, you can set up an API that fetches weather information from an external source. Setting up an API is often straightforward and can be done with just a few lines of code to request data such as the weather for a particular city. This approach increases the speed of development and makes the process much more efficient.

Prevalence and Types of APIs

APIs are **widely used in enterprise environments** because of their effectiveness in connecting different systems. They can have different structures, but the most common type is the **REST (Representational State Transfer) API**, which has become a standard for building web-based integrations due to its simplicity, scalability, and efficiency.

Use of APIs in Automation

APIs can also function as a **form of automation** since they enable applications to work together without manual effort. However, using APIs for automation requires some important considerations.

Challenges and Considerations

1. **Technical Expertise Requirement** – Developing and maintaining APIs requires people with **technical backgrounds** who understand coding and integration concepts.
2. **Development Complexity and Time** – Building an API can take time and involve **complex integration processes**. This includes not just development but also **ongoing testing** to ensure the system works correctly. Although, third-party services are available that can simplify this process.

3. **Maintenance Issues** – APIs must be maintained regularly. It is common for an API to **break when the structure changes**, meaning developers need to continuously monitor and update it.
4. **Limitations of Off-the-Shelf APIs** – Even if a ready-made API is available, there are still challenges. One major issue is **metering**, where the API usage may be limited to a certain number of requests per day or hour, unless higher pricing is paid. Additionally, APIs can still face **bugs and glitches**, especially in complex IT environments, leading to reliability concerns.

RPA as an Alternative to APIs

Due to these difficulties, **Robotic Process Automation (RPA)** has become an attractive alternative. With RPA, the **development is easier**, and there is **less need for complex integrations** compared to APIs. Interestingly, RPA platforms themselves can also act as **vehicles for delivering advanced APIs** within enterprises, creating a combination of automation power and integration capability.

Agile, Scrum, Kanban, and Waterfall

Complexity of Software Development

Software development is inherently complex, not only because of the technical challenges but also due to the need to manage teams that may be spread across different countries. Moreover, technologies continue to evolve, and maintaining a system often requires adding new features and upgrading underlying components. Even top-tier companies face difficulties; for instance, Microsoft's **Vista operating system** took five years to develop. This delay was caused by organizational silos, poor collaboration, and a lack of system-wide integration. Teams often built efficient pieces of code that failed to function properly within the larger system, leading to slow development and persistent mishaps.

The Need for Software Management Approaches

With new platforms such as the cloud and hybrid cloud making software development even more difficult, structured software management approaches have become essential.

Agile Methodology

Agile was created in the 1990s and was formally shaped by the **Manifesto for Agile Software Development**. The main focus of Agile is **incremental and iterative development**, beginning with a clear plan but relying on constant collaboration and communication among teams, including people from the business side. Modern collaboration tools such as **Slack** and **Zoom** have made Agile easier to adopt, reducing reliance on emails. For example, Tim Tully, CTO of Splunk, highlighted how his email volume declined substantially because his teams primarily collaborate through Slack.

Scrum

Scrum is a **subset of Agile** that emphasizes short, iterative cycles called **sprints**, typically lasting one to two weeks. This approach maintains project momentum and helps break down large projects into manageable portions. Interestingly, Scrum was initially used in manufacturing before being adapted successfully to software development.

Kanban

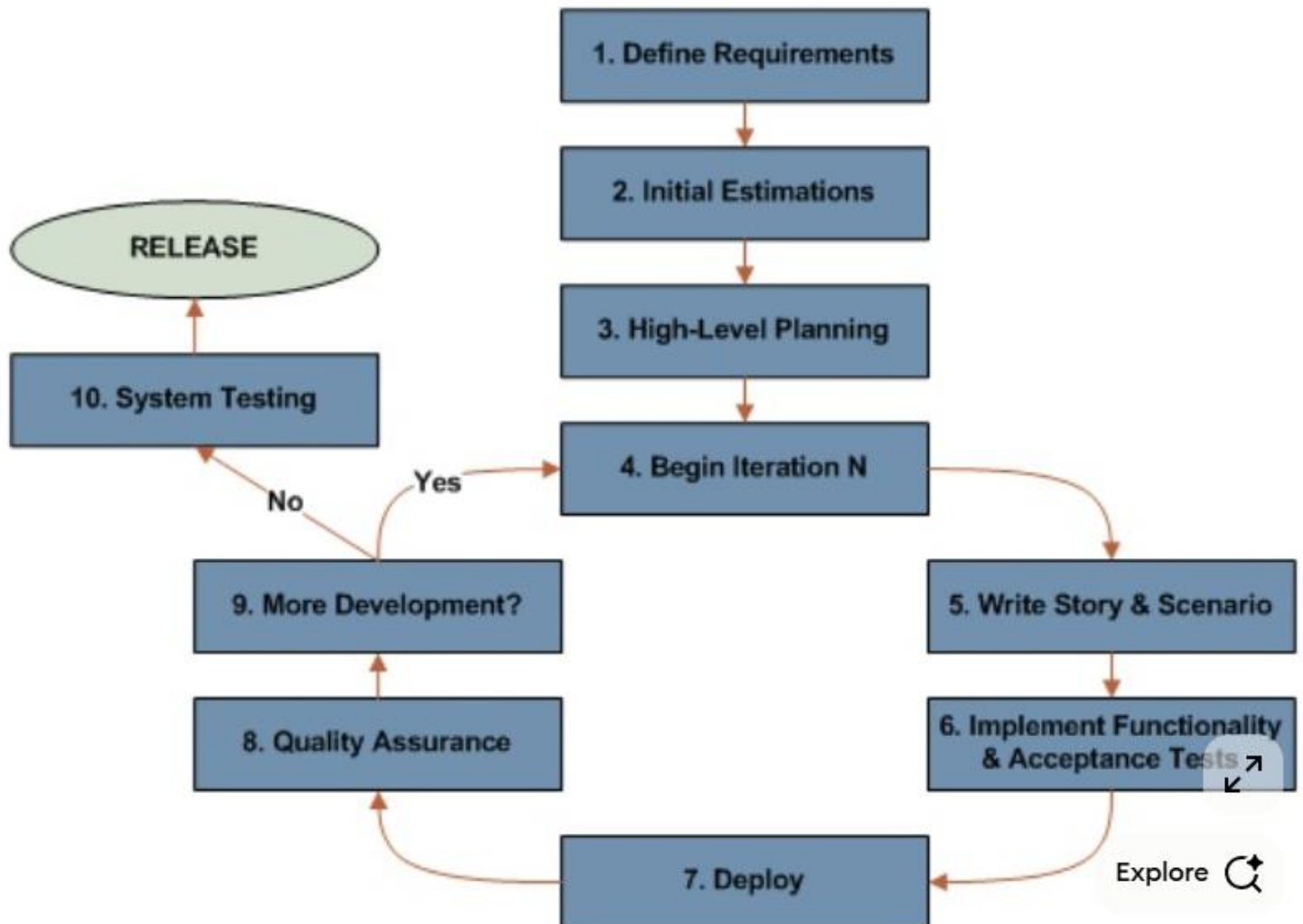
Kanban, a term derived from the Japanese word for “visual sign” or “card,” originates from **Toyota's high-quality manufacturing processes**. The system uses **visual tools** to streamline workflows and is similar to Agile in its reliance on iterative development. Kanban provides transparency and efficiency by making project progress and bottlenecks easily visible.

Waterfall

Waterfall is the **traditional software development model** dating back to the 1970s. It follows a highly structured, step-by-step plan with extensive documentation. Tools like

Gantt charts are often used to track progress. While Waterfall has the advantage of detailed planning, it has lost popularity due to several drawbacks: difficulty in making changes, tedious processes, and a high risk of delays.

Agile: Flow Diagram



Waterfall-Model Software Engineering

Requirements

Design

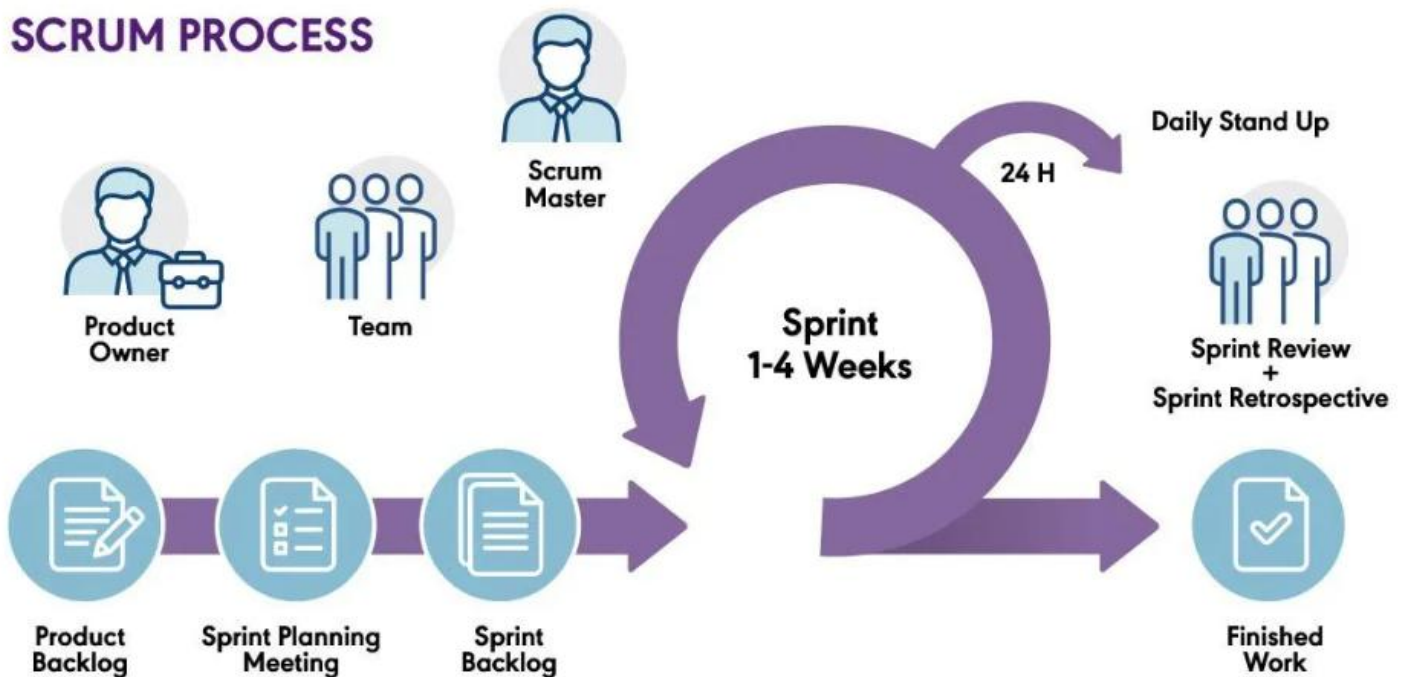
Development

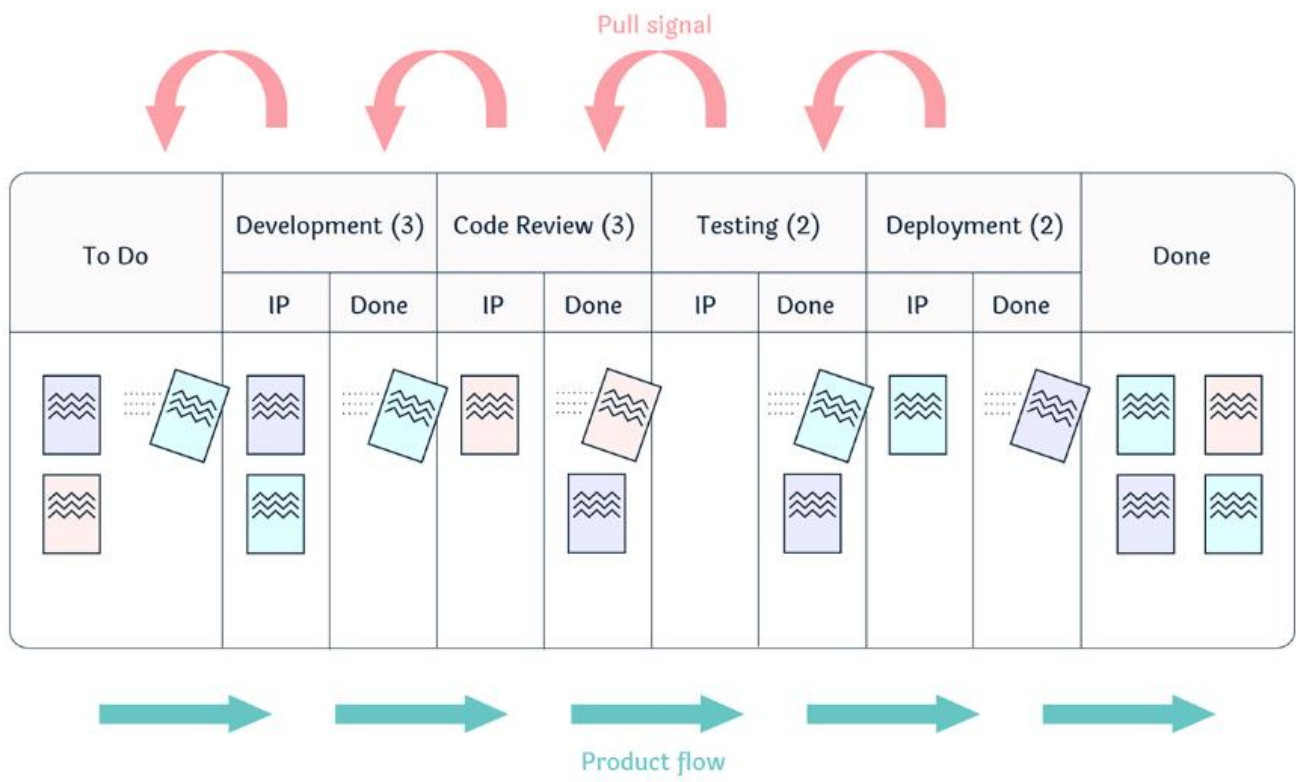
Testing

Deployment

Maintenance

SCRUM PROCESS





Kanban Chart.

